Model Tuning

ADS 503 Final Team Project - Diabetes Health Indicators (BRFSS 2015)

Vince Garcia

2026-06-20

Table of contents

```
library(tidyverse)
library(caret)
library(recipes)
library(pROC)
library(scales)
library(doParallel)
library(glmnet)
library(ranger)
library(gbm)
library(PRROC) # Explicitly loaded for Precision-Recall evaluation metrics

set.seed(503)
dm_palette <- c("No Diabetes" = "#4C9F70", "Diabetes" = "#D1495B")
```

PreProcessing from Model Training Portion

```
setwd(r"(C:\Users\User\ADS503_Team5\Raw Data)")
if (!exists("train_processed") || !exists("test_processed")) {

  df <- read_csv("diabetes_binary_5050split_health_indicators_BRFSS2015.csv") |>
    mutate(Diabetes_binary = factor(
      Diabetes_binary, levels = c(0, 1), labels = c("No Diabetes", "Diabetes")
    ))

  set.seed(503)
  split_index <- createDataPartition(df$Diabetes_binary, p = 0.80, list = FALSE)
```

```
  train_data  <- df[split_index, ]
  test_data   <- df[-split_index, ]

  diabetes_recipe <- recipe(Diabetes_binary ~ ., data = train_data) |>
    step_zv(all_predictors()) |>
    step_normalize(all_numeric_predictors())

  diabetes_prep   <- prep(diabetes_recipe, training = train_data)
  train_processed <- bake(diabetes_prep, new_data = train_data)
  test_processed  <- bake(diabetes_prep, new_data = test_data)
}

# Caret-safe target coding function
recode_target <- function(d) {
  d |>
    mutate(Diabetes_binary = factor(
      ifelse(as.character(Diabetes_binary) == "Diabetes", "Diabetes", "NoDiabetes"),
      levels = c("Diabetes", "NoDiabetes")
    ))
}

# Generate the missing train_cv and test_cv dataframes
train_cv <- recode_target(train_processed)
test_cv  <- recode_target(test_processed)

# Establish your cross-validation folds using the newly created target
set.seed(503)
cv_index <- createFolds(train_cv$Diabetes_binary, k = 5, returnTrain = TRUE)

ctrl <- trainControl(
  method          = "cv",
  index           = cv_index,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  verboseIter     = FALSE
)
#Following the same pre-processing workflow as MT Portion.qmd
```

Training data for model tuning

```r
# gemini suggested separating these into x and y matrices, removing class name Diabetes_binar
train_x <- as.matrix(train_cv[, colnames(train_cv) != "Diabetes_binary"])
test_x  <- as.matrix(test_cv[, colnames(test_cv) != "Diabetes_binary"])

train_y <- as.factor(train_cv$Diabetes_binary)
test_y  <- as.factor(test_cv$Diabetes_binary)

# have to use predict in front of preProcess according to gemini, using zv, center, and scale
x_train_processed <- predict((preProcess(train_x, method = c("zv", "center", "scale"))), trai
x_test_processed  <- predict((preProcess(train_x, method = c("zv", "center", "scale"))), test

# source for preProcess logic:
# https://www.rdocumentation.org/packages/caret/versions/6.0-92/topics/preProcess

# gemini suggested using as.data.frame for x datasets when running tree-based models like ran
x_train_df <- as.data.frame(x_train_processed)
x_test_df  <- as.data.frame(x_test_processed)
```

Baseline Logistic Regression

```r
# Baseline logistic regression floor
#Chapter 12 of Applied Predictive Modeling, using train() instead of glm()
# source for train logic:
#https://www.rdocumentation.org/packages/caret/versions/4.33/topics/train

set.seed(503)
mod_glm <- train( #use of train() allows for use of trControl = ctrl, as suggested by Gemini
  x         = x_train_df, #training from x data, to prevent data leakage
  y         = train_y,  #y training
  method    = "glm",      #generalized linear model
  family    = binomial,
  metric    = "ROC",
  trControl = ctrl
)
best_blr <- mod_glm$results
```

Elastic Net Hyperparameter Optimization

```r
# Elastic net tuning optimization, discussed in Ch. 12 of Applied Predictive modeling
# source for train logic:
#https://www.rdocumentation.org/packages/caret/versions/4.33/topics/train
```

```
# Where 'glm' and 'glmnet' was searched.

# spin up multiple cores to process the 100-grid combinations efficiently
cl <- makePSOCKcluster(detectCores() - 1) #suggested by Gemini
registerDoParallel(cl)#suggested by Gemini

set.seed(503)
mod_glmnet <- train(
  x          = x_train_df,
  y          = train_y,
  method     = "glmnet", #elastic net for generalized linear model
  metric     = "ROC",
  trControl  = ctrl,
  tuneLength = 10
)

stopCluster(cl)#suggested by Gemini
registerDoSEQ()#suggested by Gemini

best_glmnet <- mod_glmnet$results
```

Random Forest - hyperparameter tuning

```
set.seed(503)

# source for createDataPartition() logic:
# https://www.rdocumentation.org/packages/caret/versions/7.0-1/topics/createDataPartition
tune_index <- createDataPartition(train_cv$Diabetes_binary, p = 0.20, list = FALSE)
#cross validation on training data for Random Forest
x_tune_df <-as.data.frame(train_cv[tune_index, colnames(train_cv) !="class"])
#putting the class names back in for the y data
y_tune     <- train_cv$Diabetes_binary[tune_index]
#cross validation for tune_ctrl
tune_ctrl <- trainControl(
  method          = "cv",
  number          = 3, #suggested by Gemini
  classProbs      = TRUE,
  summaryFunction = twoClassSummary
)
#source for use of "ranger"
#https://www.rdocumentation.org/packages/ranger/versions/0.16.0/topics/ranger
#passing into train() instead for hyperparameter tuning
```

```r
set.seed(503)

mod_rf_tune <- train(
  x          = x_tune_df,
  y          = y_tune,
  method     = "ranger",
  metric     = "ROC",
  trControl  = tune_ctrl,
  tuneLength = 3,
  num.trees  = 150,
  importance = "impurity",
  verbose    = FALSE
)

best_hp <- mod_rf_tune$bestTune
```

Random Forest - Final Model

```r
set.seed(503)

mod_rf <- train(
  x          = x_train_df,
  y          = train_y,
  method     = "ranger",
  metric     = "ROC",
  trControl  = ctrl,
  num.trees  = 300, #twice as many trees
  importance = "impurity",
  tuneGrid   = best_hp, #suggested by Gemini, to take the result
  #of the optimal model of the hyper-parameter
  verbose    = FALSE
)
```

Saving the best model for random forest

```r
best_rf <- mod_rf$results
```

Stochastic Gradient Boosting (GBM)

Final model, GBM, hyperparameter tuning

Exporting the models to model_evaulation

```
# Set working directory to ensure it saves to a project data folder
setwd(r"(C:\Users\User\ADS503_Team5\Raw Data)")
# Save all trained models and preprocessed test objects for the evaluation file
save(
  mod_glm, mod_glmnet, mod_rf, mod_gbm, x_train_df,
  x_test_processed, x_test_df, test_y,
  file = "tuned_models_and_data.RData"
)
```